

EXPERIMENT - 5

AIM:

To create SQL views for department salary summary and employee hierarchy, test the updatability of views, and implement a recursive CTE to display reporting chains.

OBJECTIVES:

- To create a view that summarizes employee salary information department-wise.
- To create an employee hierarchy view using manager relationships.
- To test whether a view is updatable and understand the conditions affecting view updates.
- To implement a recursive CTE for displaying employee reporting chains.
- To verify the results using SELECT queries.

DATABASE USED:

The queries use the CompanyDB database and the Employee–Department–Project schema from the reference experiment. The Employee table is extended with manager_id to support employee hierarchy and recursive reporting chains.

USE CompanyDB;

SCHEMA REFERENCE:

The Department, Project and Employee tables remain connected through dept_id and project_id. The Employee table additionally contains manager_id, which references Employee(emp_id).

SOURCE CODE / SQL QUERIES

1. DEPARTMENT SALARY SUMMARY VIEW

Create a view showing employee count, average salary, minimum salary, maximum salary and total salary for each department.

```
CREATE OR REPLACE VIEW department_salary_summary AS
SELECT
    d.dept_id,
    d.dept_name,
    COUNT(e.emp_id) AS employee_count,
    AVG(e.salary) AS average_salary,
    MIN(e.salary) AS minimum_salary,
    MAX(e.salary) AS maximum_salary,
    SUM(e.salary) AS total_salary
FROM Department d
JOIN Employee e
    ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name;
```

Output:

 109 ms 

dept_id	dept_name	employee_count	average_salary	minimum_salary	maximum_salary	total_salary
1	IT	6	67833.333333	58000.00	81000.00	407000.00
2	HR	6	53666.666667	47000.00	61000.00	322000.00
3	Finance	6	69166.666667	56000.00	83000.00	415000.00
4	Marketing	6	60166.666667	51000.00	70000.00	361000.00
5	Operations	6	66500.000000	59000.00	76000.00	399000.00

The view returns one summary row for each of the five departments.

2. EMPLOYEE HIERARCHY VIEW:

Create a view containing each employee and the name of the employee's immediate manager.

```
CREATE OR REPLACE VIEW employee_hierarchy AS
SELECT
    e.emp_id,
    e.emp_name,
    e.dept_id,
    d.dept_name,
    e.manager_id,
    m.emp_name AS manager_name,
    e.salary
FROM Employee e
JOIN Department d
    ON e.dept_id = d.dept_id
```

```
LEFT JOIN Employee m
ON e.manager_id = m.emp_id;
```

Output:

Output:

emp_id	emp_name	dept_id	dept_name	manager_id	manager_name	salary
1	Aarav	1	IT	NULL	NULL	65000.00
2	Vivaan	1	IT	1	Aarav	72000.00
3	Aditya	1	IT	1	Aarav	58000.00
4	Arjun	1	IT	1	Aarav	81000.00
5	Kabir	1	IT	1	Aarav	69000.00
6	Reyansh	1	IT	1	Aarav	62000.00
7	Ananya	2	HR	NULL	NULL	52000.00
8	Diya	2	HR	7	Ananya	48000.00
9	Myra	2	HR	7	Ananya	61000.00
10	Sara	2	HR	7	Ananya	55000.00
11	Ishita	2	HR	7	Ananya	47000.00
12	Meera	2	HR	7	Ananya	59000.00
13	Rohan	3	Finance	NULL	NULL	75000.00
14	Karan	3	Finance	13	Rohan	68000.00
15	Nikhil	3	Finance	13	Rohan	83000.00
16	Yash	3	Finance	13	Rohan	62000.00
17	Manav	3	Finance	13	Rohan	71000.00
18	Dev	3	Finance	13	Rohan	56000.00
19	Aanya	4	Marketing	NULL	NULL	54000.00
20	Kiara	4	Marketing	19	Aanya	63000.00
21	Tanya	4	Marketing	19	Aanya	57000.00
22	Riya	4	Marketing	19	Aanya	66000.00
23	Avni	4	Marketing	19	Aanya	51000.00
24	Navya	4	Marketing	19	Aanya	70000.00
25	Samar	5	Operations	NULL	NULL	60000.00
26	Dhruv	5	Operations	25	Samar	73000.00
27	Atharv	5	Operations	25	Samar	67000.00
28	Parth	5	Operations	25	Samar	59000.00
29	Rudra	5	Operations	25	Samar	76000.00
30	Veer	5	Operations	25	Samar	64000.00

The hierarchy view shows the immediate reporting relationship. A department head has NULL in manager_id and manager_name.

3. TEST UPDATABILITY OF VIEWS:

Test an update on a simple employee hierarchy view and then test an update on the grouped department salary summary view.

```
UPDATE employee_hierarchy
SET salary = salary + 1000
WHERE emp_id = 6;
```

```

SELECT emp_id, emp_name, salary
FROM Employee
WHERE emp_id = 6;

UPDATE department_salary_summary
SET average_salary = average_salary + 1000
WHERE dept_id = 1;

```

Output:

The first UPDATE is expected to fail because the employee_hierarchy view contains joins and does not directly expose a single base-table row for every selected column. The grouped department_salary_summary view is also not directly updatable because it contains GROUP BY and aggregate functions. The base Employee table should be updated instead.

4. RECURSIVE CTE – REPORTING CHAIN:

Display the reporting chain beginning with department heads and recursively follow manager relationships.

```

WITH RECURSIVE reporting_chain AS (
    SELECT
        e.emp_id,
        e.emp_name,
        e.manager_id,
        1 AS level,
        CAST(e.emp_name AS CHAR(500)) AS reporting_path
    FROM Employee e
    WHERE e.manager_id IS NULL

    UNION ALL

    SELECT
        e.emp_id,
        e.emp_name,
        e.manager_id,
        rc.level + 1,
        CONCAT(rc.reporting_path, ' -> ', e.emp_name)
    FROM Employee e
    JOIN reporting_chain rc
        ON e.manager_id = rc.emp_id
)
SELECT emp_id, emp_name, manager_id, level, reporting_path
FROM reporting_chain
ORDER BY reporting_path;

```

Output:

   AKSHAT PANDEY

I/O AI Agent 101 ms 

STDIN

Input for the program (Optional)

Output:

emp_id	emp_name	manager_id	level	reporting_path
19	Aanya	NULL	1	Aanya
23	Avni	19	2	Aanya -> Avni
20	Kiara	19	2	Aanya -> Kiara
24	Navya	19	2	Aanya -> Navya
22	Riya	19	2	Aanya -> Riya
21	Tanya	19	2	Aanya -> Tanya
1	Aarav	NULL	1	Aarav
3	Aditya	1	2	Aarav -> Aditya
4	Arjun	1	2	Aarav -> Arjun
5	Kabir	1	2	Aarav -> Kabir
6	Reyansh	1	2	Aarav -> Reyansh
2	Vivaan	1	2	Aarav -> Vivaan
7	Ananya	NULL	1	Ananya
8	Diya	7	2	Ananya -> Diya
11	Ishita	7	2	Ananya -> Ishita
12	Meera	7	2	Ananya -> Meera
9	Myra	7	2	Ananya -> Myra
10	Sara	7	2	Ananya -> Sara
13	Rohan	NULL	1	Rohan
18	Dev	13	2	Rohan -> Dev
14	Karan	13	2	Rohan -> Karan
17	Manav	13	2	Rohan -> Manav
15	Nikhil	13	2	Rohan -> Nikhil
16	Yash	13	2	Rohan -> Yash
25	Samar	NULL	1	Samar
27	Atharv	25	2	Samar -> Atharv
26	Dhruv	25	2	Samar -> Dhruv
28	Parth	25	2	Samar -> Parth
29	Rudra	25	2	Samar -> Rudra
30	Veer	25	2	Samar -> Veer

Light 

RESULT:

Views were created for department salary summaries and employee hierarchy. View update behavior was tested, and a recursive CTE was used successfully to display reporting chains.

CONCLUSION:

SQL views provide reusable logical representations of data, while recursive CTEs can traverse self-referencing employee relationships to display hierarchical reporting structures.